WHAP Implementation Plan

Zero-cost build plan for the Wagers Hiring Application Platform (Supabase + React PWA) | Version 1.0 | November 2026

1. What we are building

WHAP is a hyperlocal micro-task marketplace. A requester posts a small chore priced between Rs 50 and Rs 300. Only workers within 1.5 km can see it and send price bids (reverse bidding). The requester accepts one bid, and the agreed amount moves from the requester's wallet into a simulated escrow. When the work is done, the requester shows a 4-digit OTP, the worker enters it, and the escrow is released to the worker minus a platform fee. Both sides then review each other, and reviews stay hidden until both have submitted (double-blind).

This document is a step-by-step plan that any team member can follow to build the application from an empty folder, using free tools and an AI coding agent.

2. Goals and constraints

- Zero cost: every tool and service must have a free plan.
- Handles about 1,000 registered users and about 200 users online at the same time.
- Every core feature works correctly, proven by an automated test script.
- Runs on low-spec laptops (8 GB RAM): no emulators, no heavy local servers.
- Mobile-first web app that can be installed on a phone from the browser (PWA).
- Escrow is simulated with ledger tables. No real money, no payment gateway.

3. Tech stack

- Database, login and live updates: Supabase (PostgreSQL with the PostGIS extension, Auth, Realtime).
- Business logic: SQL functions inside the database, each running as one transaction.
- Security: Row Level Security (RLS) on every table.
- Frontend: React, Vite, Tailwind CSS, react-router, lucide-react, react-leaflet (OpenStreetMap), vite-plugin-pwa. JavaScript, not TypeScript.
- Hosting: a free static host (Vercel, Netlify or Cloudflare Pages). Check their current free limits.

- Tooling: Git and GitHub, Node.js 20.19 or newer, OpenSpec (spec-driven workflow), an AI coding agent such as Antigravity.

Free-tier facts to verify on supabase.com/pricing before you start: about 500 MB database storage, about 200 concurrent realtime connections, 50,000 monthly active users, two free projects, and projects pause after 7 days of inactivity. These limits change over time.

4. Before you start

- Install Git, Node.js 20.19 or newer and a code editor or AI IDE.
- Create a free Supabase account and a new project. Save the database password.
- Create a free GitHub account and an empty repository.
- Keep the project folder outside OneDrive or any synced folder, for example D:\Projects\whap.
- Close heavy apps while building. Run one heavy task at a time.

5. Suggested team roles (for four builders)

- Builder 1, Database: tables, RLS, seed data (Phases 2 and 3).
- Builder 2, Logic and tests: database functions and the Node test script (Phases 3 and 6).
- Builder 3, Requester app: requester screens (Phases 4 and 5).
- Builder 4, Worker app and docs: worker screens, deployment and documentation (Phases 5, 7 and 8).

One person owns the main branch. Everyone else works on branches and merges through pull requests.

6. Business rules

Keep these as configurable values, not hard-coded numbers.

- Search radius: 1.5 km.
- Task budget: Rs 50 to Rs 300.
- Bid price: at least Rs 20 and at most 2 times the task budget. One bid per worker per task (a new bid updates the old one while it is pending).
- Platform fee: 10% of the agreed price, minimum Rs 5, taken from the worker payout.
- Bidding window: 3 minutes. A task expires after 30 minutes with no accepted bid.
- OTP: 4 digits, valid for 2 hours. After 3 wrong attempts the task becomes DISPUTED.

- Limits: at most 3 open tasks per requester, at most 1 active job per worker.
- Wallet: requesters start with a Rs 500 demo balance. A wallet balance can never go below 0.
- Money uses exact decimal numbers. Statuses use fixed lists of values.

State rules (any other change must be rejected):

- Task: OPEN to ASSIGNED to COMPLETED. OPEN to CANCELLED or EXPIRED. ASSIGNED to CANCELLED (full refund) or DISPUTED. DISPUTED to COMPLETED or CANCELLED (admin only).
- Bid: PENDING to ACCEPTED, REJECTED or WITHDRAWN.
- Escrow: HELD to RELEASED or REFUNDED, and never reversed.

Security rules:

- Only the task's requester can read the OTP.
- Tasks beyond 1.5 km are never returned to a worker, even when asked for by id.
- Wallet and escrow tables can only be changed through the database functions, never directly from the app.
- Never put the Supabase service-role key in the frontend or in Git.

7. Data model

- profiles: id (linked to the login user), name, role (REQUESTER, WORKER or ADMIN), location, rating average, rating count.
- wallets: user id, balance (never below 0).
- tasks: id, requester id, title, description, category, budget, location (PostGIS geography point), address, status, assigned worker id, agreed price, OTP code, OTP attempts, OTP expiry, version, created and completed times.
- bids: id, task id, worker id, price, status, created time.
- escrow_transactions: id, task id, amount, platform fee, status, created and released times.
- ledger_entries: id, wallet id, type (TOPUP, ESCROW_LOCK, ESCROW_REFUND, PAYOUT, FEE), amount, task id, created time.
- reviews: id, task id, reviewer id, reviewee id, rating from 1 to 5, comment, visible flag.
- notifications: id, user id, message, read flag, created time.

Categories: XEROX_PICKUP, BUY_ITEMS, CARRY_THINGS, QUEUE, OTHER. Add a GiST index on the task location and indexes on all foreign keys and status columns.

8. Database functions (the application logic)

Each function runs in one transaction and is called from the app with supabase-js rpc.

- post_task: validates the budget and the open-task limit, then creates the task.
- nearby_tasks: returns open tasks within 1.5 km of the worker, sorted by distance, with a distance value. Uses ST_DWithin.
- place_bid: checks that the task is open, the bidder is within 1.5 km, the price is valid, and the bidder is not the requester.
- accept_bid: locks the task row, checks the task is open and the bid is pending, checks the wallet balance, moves the money to escrow (HELD), marks this bid ACCEPTED and all others REJECTED, sets the task to ASSIGNED and creates the OTP. Must be safe when two accepts happen at the same time.
- complete_task: checks the worker and the OTP. A correct OTP releases the escrow, pays the worker minus the fee and completes the task. A third wrong attempt sets the task to DISPUTED.
- cancel_task: allowed while OPEN, or while ASSIGNED with a full escrow refund.
- resolve_dispute: admin only. Either refunds the requester or releases the payout to the worker.
- submit_review: allowed after completion, one review per side. Reviews become visible when both sides have submitted, or after 7 days, and the worker rating is updated.
- expire_tasks: marks old open tasks as EXPIRED. Run it as a scheduled job, or check it when tasks are read.

9. Implementation phases

Phase 0: Prepare (about 1 hour)

- Do the setup in section 4.
- Done when: the project folder exists, the Git repository is created, and you have the Supabase project URL and anon key.

Phase 1: Specs and rules (about half a day)

- Run openspec init and choose your AI tool.
- Write the project rules file (AGENTS.md) and the OpenSpec specs: product, architecture, rules, data model, API (the database functions), client screens and seed data.
- Create a list of changes, one per feature, in the build order below.
- Done when: openspec validate passes and the whole team has read the rules.

Phase 2: Database foundation (about 1 day)

- Create the tables, PostGIS extension, indexes and RLS policies as SQL migration files, managed with the Supabase command line.
- Add seed data: 2 requesters, 4 workers, 1 admin, funded wallets, and 8 tasks around a sample campus, with 6 inside 1.5 km and 2 outside.
- Done when: the tables are visible in the Supabase dashboard and a test query confirms tasks beyond 1.5 km are hidden from a worker.

Phase 3: Business logic (about 1.5 days)

Build the functions in this order, adding a test for each to the Node test script:

- post_task and nearby_tasks.
- place_bid.
- accept_bid.
- complete_task.
- cancel_task.
- resolve_dispute, submit_review and expire_tasks.

Done when: the test script passes the full flow and every edge case in section 10.

Phase 4: Frontend foundation (about 1 day)

- Set up Vite and React, Tailwind, routing and the design system.
- Build register and login, and protected routes for each role.
- Add the PWA manifest and service worker.
- Done when: a requester and a worker can sign in, in two browser windows, and each sees their own interface.

Phase 5: Requester and worker screens (about 2 days)

Requester screens:

- Home with a map and the 1.5 km circle.
- Post task in 3 steps (details, location, budget).
- Task screen with live bids and an Accept button, an escrow strip and a progress tracker.
- OTP card, wallet, and review screen.

Worker screens:

- Nearby jobs sorted by distance, and a bid sheet with a price stepper.

- My Bids (pending, accepted, lost).
- Active job with four OTP digit boxes and a Complete button.
- Earnings and review screen.

Live updates use Supabase Realtime, with a 3-second polling fallback. Done when: two accounts can run the whole story, from posting a task to the worker getting paid.

Phase 6: Testing and hardening (about 1 day)

- Run the full Node test script and the concurrency test (see section 10).
- Test on a real phone on the same Wi-Fi.
- Try to break it: read another user's data, read the OTP as a worker, change a wallet directly.
- Done when: all tests pass and none of the break attempts work.

Phase 7: Deploy for free (about half a day)

- Push the code to GitHub and deploy the frontend to the free static host.
- Set the Supabase URL and anon key as environment variables on the host.
- Open the live link on a phone and use Add to Home screen.
- Add a scheduled GitHub Action that calls the API every few days, so the free Supabase project does not pause from inactivity. Confirm this works with Supabase's current rules.
- Done when: a friend can open the link and complete a task from start to finish.

Phase 8: Documentation (about half a day)

- Write README.md with setup and run steps.
- Write PROJECT_EXPLAINED.md: each table, each function and each screen in simple English.
- Prepare a 3-minute demo script and practise it twice.

10. Testing checklist

The Node test script (using supabase-js) must create test users and check wallet and escrow balances after every step.

- Full happy path: post, bid, accept, correct OTP, payout, balances correct.
- Accept with too little wallet balance is rejected.
- Two accepts at the same time: only one succeeds.
- Bid from outside 1.5 km is rejected.

- Bid below the minimum or above 2 times the budget is rejected.
- Wrong OTP three times sets the task to DISPUTED.
- Cancel after accept refunds the escrow in full.
- A worker cannot read the OTP. A user cannot read another user's wallet.
- The wallet balance never goes below 0.
- Reviews stay hidden until both sides submit.
- Concurrency check: 100 simultaneous requests to nearby_tasks, with response times reported.

11. Working with an AI coding agent

- Give the agent the rules file first and ask it to re-read it before every task.
- Work one change at a time, and ask for the real command output after every change. Never accept a claim of success without output.
- Do not let the agent silently swap a tool for another one. If something is missing, it must stop and say so.
- Never delete or weaken a test to make it pass.
- Commit after every working change so you can go back.
- If the agent loops on an error, stop it, ask for the root cause in plain words, then agree on one fix.
- Read what the agent writes. You must be able to explain every table, function and screen.

12. Limits and risks

- Escrow is simulated. Real payments need a licensed provider, so the app must say Prototype - simulated payments in Wallet or Profile.
- The free Supabase project pauses after a week of inactivity, so keep it alive and check it before any demo.
- About 200 concurrent realtime connections is the free-plan ceiling. Enough for a campus, not for a city.
- Location comes from the user's device, and spoofing is not handled.
- Out of scope: real money, identity checks, chat, voice, photo upload, push notifications, native Play Store app.
- Free-tier limits change. Re-check pricing pages before relying on any number in this document.

13. If time runs short

Cut in this order, and never cut the first three items of the core flow:

- Core, never cut: accept bid with escrow, OTP completion, wallet ledger.
- Cut first: extra polish and animations.
- Cut next: the PWA install step (the website still works).
- Cut next: dispute screens (keep cancel and refund).
- Cut next: reviews and notifications.

14. Final checklist

- All tests pass and the output is saved.
- A real phone can run the full flow.
- RLS verified: no user can see data that is not theirs.
- No secrets in Git, and the service-role key is not in the frontend.
- The live link works, and the keep-alive is set up.
- README.md and PROJECT_EXPLAINED.md are written.
- The demo script is practised, and the team agrees on what is built and what is only planned.

Appendix: Starter prompt for the AI agent

Paste this into an empty project folder, after openspec init:

You are building WHAP, a hyperlocal micro-task marketplace, using the OpenSpec workflow. Stack: Supabase (Postgres with PostGIS, Auth, Realtime, Row Level Security), SQL functions for all business logic as single transactions, and a React, Vite, Tailwind PWA frontend (mobile-first, separate requester and worker interfaces). Use the rules, data model and functions described in the implementation plan. Never invent versions, APIs or commands, never silently substitute a tool, and show real output after every change. Do not run the local Supabase Docker stack. Stop and tell me when you need me to create the Supabase project, copy the URL and anon key, or log in to the Supabase command line. First, create the specs and the change plan, run openspec validate, and wait for my approval.